

Megatron-LM & Megatron Core

Complete Reference Notes

Architecture · Parallelism · Training · Fine-tuning · MoE · FP8 · Production Workflows

TP · SP · PP · CP · EP · DP · VPP · Interleaved 1F1B · Distributed Optimizer · FP8 · MoE · Context
Parallelism · FSDP · HSDP · NeMo · DeepSpeed · TRT-LLM · BLOOM · MT-NLG 530B · Nemotron ·
DeepSeek

Sources: NVIDIA/Megatron-LM GitHub · Megatron Core Docs · arXiv papers · Production Case Studies ·
Megatron Bridge Docs

1. Overview & Ecosystem

1.1 What Is Megatron-LM?

Megatron-LM is NVIDIA's open-source framework for training massive transformer language models on GPU clusters. Released in 2019 with the groundbreaking paper introducing tensor parallelism for transformers, it has since become the industry standard backbone for training frontier-scale LLMs. The framework has two tightly coupled components: Megatron-LM (the reference training layer) and Megatron Core (the reusable GPU-optimised library with formal API versioning).

Megatron Core achieves up to 47% Model FLOP Utilization (MFU) on H100 clusters — among the highest publicly reported numbers. It has been used to train models from 2B to 530B+ parameters, and its techniques underpin landmark models including MT-NLG 530B, BLOOM 176B, and multiple internal NVIDIA/Microsoft frontier models.

1.2 Two-Layer Architecture

Component	Role & Best Used For
Megatron-LM (megatron/)	Reference training framework with pre-configured scripts for GPT, BERT, T5, Mamba, MoE. Minimal abstraction, full customisability. Best for researchers and learning.
Megatron Core (megatron/core/)	Composable library with versioned APIs. GPU-optimised building blocks. Best for ML engineers building custom training pipelines or integrating with NeMo/DeepSpeed.
Megatron Bridge	Bidirectional HuggingFace <-> Megatron checkpoint conversion. Production-ready model recipes for popular architectures.
NVIDIA NeMo	Enterprise cloud-native AI platform built on Megatron Core. End-to-end: pre-train, SFT, RLHF, deploy. Full managed infrastructure support.
TRT-LLM Export	Export Megatron Core models to TensorRT-LLM for highest-performance inference on NVIDIA hardware. FP8/INT8/INT4 quantisation.

1.3 Repository Structure

NVIDIA/Megatron-LM/	
├─ megatron/	
│ └─ core/	# Megatron Core — formal library
│ └─ models/	# GPTModel, BertModel, MambaModel, T5Model
│ └─ transformer/	# TransformerLayer, TransformerBlock, Attention, MLP,
MoE	
│ └─ tensor_parallel/	# ColumnParallelLinear, RowParallelLinear, mappings
│ └─ pipeline_parallel/	# schedules (1F1B, interleaved 1F1B), P2P comms
│ └─ distributed/	# DDP, FSDP, HSDP, grad buffers
│ └─ optimizer/	# DistributedOptimizer, MixedPrecisionOptimizer
│ └─ datasets/	# GPTDataset, BlendedMegatronDatasetBuilder,
IndexedDataset	

			inference/	# DynamicInferenceEngine, StaticInferenceEngine, KV
	cache			
			dist_checkpointing/	# Distributed checkpoint save/load
			fusions/	# Fused kernels: LayerNorm, softmax, bias+GeLU, SwiGLU
			extensions/	# Transformer Engine (FP8), Kitchen backend
			quantization/	# FP8/FP4 quantisation utilities
			export/	# TRT-LLM export utilities
			training/	# Training loop, arg parser, yaml config, timers
			pretrain_gpt.py	# GPT pre-training entry point
			pretrain_bert.py	# BERT pre-training entry point
			pretrain_t5.py	# T5 pre-training entry point
			examples/	# Scripts for GPT-3 175B, Llama, MoE, multimodal
			tools/	
			preprocess_data.py	# Tokenise + binary index raw text data
			convert_checkpoint/	# Megatron <-> HuggingFace <-> DeepSpeed converters
			tests/	# Unit, integration, and regression tests

1.4 Key Research Papers

Paper	Contribution
Megatron-LM (2019, arXiv:1909.08053)	Introduced tensor parallelism for transformers: ColumnParallelLinear / RowParallelLinear. 8.3B model on 512 V100s at 15.1 PetaFLOPs/s (76% scaling efficiency).
Efficient Large-Scale LM Training (2021, arXiv:2104.04473)	Proposed Interleaved 1F1B pipeline schedule (Virtual PP). 3D parallelism theory. 1T model at 502 PetaFLOP/s on 3072 GPUs.
Reducing Activation Recomputation (2022, arXiv:2205.05198)	Sequence Parallelism. Selective activation recomputation. 5x activation memory reduction over full recompute at near-zero compute overhead.
MT-NLG 530B (2022, NVIDIA+Microsoft)	3D parallel training: TP=8 (Megatron) + PP=35 (DeepSpeed) + ZeRO-DP on 4,480 A100 GPUs.

2. Installation & Environment Setup

2.1 Prerequisites

Requirement	Details
NVIDIA GPU	Volta+ for basic use. Ampere (A100) for BF16. Hopper (H100) for FP8 + Flash Attention v3.
CUDA	12.x recommended. Required for Transformer Engine FP8 on H100.
Python	3.8+ (3.10+ recommended)
PyTorch	2.0+ recommended (torch.compile, FSDP2 support)
NCCL	Latest version — critical for multi-node AllReduce/AllGather performance.
NVIDIA Apex	Fused LayerNorm, fused Adam. Install from source for optimised kernels.
Transformer Engine (TE)	Required for FP8 training on H100. NVIDIA's quantisation library.
FlashAttention 2	Recommended for all sequence-length training. Flash Attention 3 for H100.

2.2 NGC Container (Recommended)

```
# Pull NVIDIA PyTorch container (includes Apex, NCCL, CUDA, TE)
docker pull nvcr.io/nvidia/pytorch:24.07-py3

# Run container with all GPUs and host network
docker run --gpus all --ipc=host --network=host \
  -v /path/to/data:/workspace/data \
  -v /path/to/checkpoints:/workspace/checkpoints \
  -it nvcr.io/nvidia/pytorch:24.07-py3 bash

# Clone Megatron-LM inside container
git clone https://github.com/NVIDIA/Megatron-LM.git
cd Megatron-LM
pip install -e . # install in editable mode

# (Optional) NeMo container for enterprise features
docker pull nvcr.io/nvidia/nemo:24.07
```

2.3 Build from Source

```
git clone https://github.com/NVIDIA/Megatron-LM.git
cd Megatron-LM
# Limit parallel compilation jobs to avoid OOM during build
MAX_JOBS=4 pip install -e .

# Install NVIDIA Apex for fused kernels (strongly recommended)
git clone https://github.com/NVIDIA/apex.git
cd apex
pip install -v --disable-pip-version-check --no-cache-dir \
  --global-option='--cpp_ext' --global-option='--cuda_ext' ./
```

```
cd ..

# Transformer Engine for FP8 on H100
pip install transformer-engine[pytorch]

# Flash Attention 2
pip install flash-attn --no-build-isolation

# Optional: sentencepiece for LLaMA tokenizer
pip install sentencepiece
```

3. Parallelism Strategies — Deep Dive

Megatron-LM composes five orthogonal parallelism dimensions. Understanding their interaction is the core skill needed to configure large-scale training efficiently.

GPU Count Formula

$\text{world_size} = \text{TP} \times \text{PP} \times \text{CP} \times \text{DP}$

$\text{DP} = \text{world_size} / (\text{TP} \times \text{PP} \times \text{CP})$

EP is a sub-dimension of DP (each DP rank holds E/EP experts)

Example: 256 GPUs = $\text{TP}=8 \times \text{PP}=4 \times \text{CP}=2 \times \text{DP}=4$

Example: 128 GPUs = $\text{TP}=8 \times \text{PP}=4 \times \text{DP}=4$ (no CP)

3.1 Tensor Parallelism (TP) — Intra-Layer

Tensor Parallelism distributes individual weight matrices within a single transformer layer across multiple GPUs. The key breakthrough in the original Megatron-LM paper was pairing column-wise and row-wise splits so only one AllReduce is needed per MLP/attention block, not one per GEMM.

3.1.1 MLP Splitting Mathematics

Column + Row Pairing Pattern

Standard MLP: $Y = \text{Dropout}(\text{GeLU}(XA) * B)$

X: input [batch, seq, H] A: [H, 4H] B: [4H, H]

Column-wise split A into $[A_1|A_2|\dots|A_t]$ ($t = \text{TP degree}$):

GPU_i: $Y_i = \text{GeLU}(X * A_i)$ shape [batch, seq, $4H/t$]

No communication needed here — all GPUs have same X

Row-wise split B into $[B_1; B_2; \dots; B_t]$:

GPU_i: $Z_i = Y_i * B_i$ shape [batch, seq, H]

$Z = \text{sum_reduce}(Z_i)$ — single AllReduce across TP group

Total comm: 1 AllReduce fwd + 1 AllReduce bwd = 2 per layer

vs. naive: 2 AllReduces fwd + 2 bwd = 4 per layer

With SP: ReduceScatter + AllGather instead of 2 AllReduces

```
# Megatron Core tensor-parallel layers
from megatron.core.tensor_parallel.layers import (
    ColumnParallelLinear, # split output dimension across TP
    RowParallelLinear,   # split input dimension across TP
    VocabParallelEmbedding, # split vocabulary across TP
)
from megatron.core.tensor_parallel.mappings import (
    copy_to_tensor_model_parallel_region,
    reduce_from_tensor_model_parallel_region,
    scatter_to_tensor_model_parallel_region,
    gather_from_tensor_model_parallel_region,
)
```

```

# Column-parallel: input X is replicated; output sharded along columns
col = ColumnParallelLinear(
    input_size=1024,
    output_size=4096,
    bias=True,
    gather_output=False,      # keep output sharded for row-parallel fusion
    async_tensor_model_parallel_allreduce=True,
    sequence_parallel=True, # use ReduceScatter/AllGather instead of AllReduce
)

# Row-parallel: input sharded from col-parallel; AllReduce after
row = RowParallelLinear(
    input_size=4096,
    output_size=1024,
    bias=True,
    input_is_parallel=True, # input already sharded from ColumnParallelLinear
    sequence_parallel=True,
)

# Attention: Q,K,V are column-parallel (heads split across TP)
# Output proj: row-parallel (AllReduce across TP)
# Constraint: num_heads % TP == 0 AND hidden_size % TP == 0

```

3.1.2 TP Best Practices

Practice	Reason
TP only within NVLink node	AllReduce in every layer — needs high bandwidth. NVLink 600 GB/s vs InfiniBand 400 Gb/s (50 GB/s) = 12x faster.
TP ≤ node size (typically ≤ 8)	Beyond 8-way TP, AllReduce overhead grows faster than compute savings. Use PP for inter-node scaling.
Always enable --sequence-parallel	SP replaces TP AllReduce with ReduceScatter+AllGather. Shards activations in non-TP ops. Free memory saving.
Enable --tp-comm-overlap	Overlaps TP communication with next GEMM computation. Up to 30% TP comm latency hidden.
Validate <code>hidden_size % TP == 0</code>	Also: <code>num_attention_heads % TP == 0</code> , <code>ffn_hidden_size % TP == 0</code> .

3.2 Sequence Parallelism (SP)

Sequence Parallelism extends TP by sharding layer activations that TP does not shard (LayerNorm, Dropout inputs/outputs) along the sequence dimension. Since AllReduce = ReduceScatter + AllGather, the communication volume is identical but each GPU stores only 1/TP of the sequence-dimension activations.

SP Memory Benefit (GPT-3 example)

Config: hidden=12288, seq=2048, batch=1, TP=8

Without SP: All 8 TP GPUs hold full [2048, 12288] activations = 100 MB per GPU

With SP: Each GPU holds [256, 12288] slice only = 12.5 MB per GPU

Savings: 87.5 MB per GPU per LayerNorm/Dropout layer

Zero extra communication (ReduceScatter + AllGather = original AllReduce)

SP must be enabled with TP: `--sequence-parallel` requires `--tensor-model-parallel-size > 1`

```
# Enable SP alongside TP
python pretrain_gpt.py \
  --tensor-model-parallel-size 8 \
  --sequence-parallel \
  # SP flow in transformer layer:
  # input: [S/TP, B, H] - sharded along seq
  # AllGather before ColumnParallelLinear -> [S, B, H]
  # ColumnParallelLinear -> [S, B, 4H/TP]
  # GeLU (local)
  # RowParallelLinear -> [S, B, H/TP]
  # ReduceScatter after RowParallelLinear -> [S/TP, B, H]
  # LayerNorm on [S/TP, B, H] - each GPU only normalises its slice
```

3.3 Pipeline Parallelism (PP)

Pipeline Parallelism partitions the model's layers sequentially across GPU groups. With 96 layers and PP=8, each stage holds 12 layers. The key challenge is keeping all stages busy (reducing pipeline bubble) while minimising memory for in-flight activations.

3.3.1 Schedule Comparison

Schedule	Bubble Ratio	Memory	Best For
Naive (GPipe)	$(PP-1)/PP \approx 75\%$ for PP=4	$O(m \times PP)$ activations	Never — too much idle
1F1B (PipeDream-Flush)	$(PP-1)/(m+PP-1)$	$O(PP)$ activations	Default: good balance
Interleaved 1F1B (VPP)	$(PP-1)/(V \times m + PP - 1)$	$O(V \times PP)$ activations	High throughput, more comm
Zero-Bubble (ZBH1)	~0%	Higher (W grad buffer)	Experimental, maximises MFU

Bubble Ratio Intuition

1F1B example: PP=4, m=16 microbatches

Bubble = $(4-1) / (16+4-1) = 3/19 \approx 16\%$ (acceptable)

Interleaved 1F1B with V=2 virtual stages:

Bubble = $(4-1) / (2 \times 16 + 4 - 1) = 3/35 \approx 9\%$ (better!)

Rule: Use at least $m \geq 2 \times PP$ microbatches to keep bubble < 33%

Use $m \geq 4 \times PP$ for < 20% bubble

```
# Pipeline Parallelism configuration
python pretrain_gpt.py \
  --pipeline-model-parallel-size 8 \           # 8 pipeline stages
```

Schedule	Bubble Ratio	Memory	Best For
<pre> --num-layers 96 \ # 12 layers per stage --micro-batch-size 2 \ --global-batch-size 2048 \ # num_microbatches=128 >> PP=8 ✓ # Interleaved 1F1B (Virtual Pipeline Parallelism) --num-layers-per-virtual-pipeline-stage 2 # VPP=2: each GPU handles 2 chunks # Custom pipeline layout (for imbalanced models like DeepSeek-V3) # --pipeline-model-parallel-layout 'Et*3 (tt)*29,m L' # E=embedding, t=transformer, m=mtp_layer, L=loss # Stages split by ' ', replication by '*N' # PP P2P communication from megatron.core.pipeline_parallel import p2p_communication # Stage i sends activations to stage i+1 p2p_communication.send_forward(output_tensor, timers=timers) # Stage i+1 receives from stage i input_tensor = p2p_communication.recv_forward(timers=timers) # Backward gradient flow (reverse direction) p2p_communication.send_backward(grad, timers=timers) grad_output = p2p_communication.recv_backward(timers=timers) </pre>			

3.3.2 forward_backward_func API

```

from megatron.core.pipeline_parallel.schedules import get_forward_backward_func

forward_backward_func = get_forward_backward_func()

# User-defined forward step
def forward_step(data_iterator, model):
    data = next(data_iterator)
    tokens = data['tokens'].cuda()
    labels = data['labels'].cuda()
    loss_mask = data['loss_mask'].cuda()
    attention_mask = data['attention_mask'].cuda()
    position_ids = data['position_ids'].cuda()

    output_tensor = model(tokens, position_ids, attention_mask, labels=labels)

    # Return (output, loss_func) tuple
    def loss_func(output_tensor):
        losses = output_tensor.float()
        loss = torch.sum(losses.view(-1) * loss_mask.view(-1).float())
        loss /= loss_mask.sum()
        return loss, {'lm loss': loss}

    return output_tensor, loss_func

# Execute forward-backward with pipeline schedule
losses = forward_backward_func(
    forward_step_func=forward_step,
    data_iterator=train_data_iterator,
    model=model, # list of stage modules for interleaved PP
    num_microbatches=num_microbatches,

```

```

    seq_length=args.seq_length,
    micro_batch_size=args.micro_batch_size,
    forward_only=False,      # True for evaluation/inference only
)

```

3.4 Context Parallelism (CP)

Context Parallelism shards ALL layer activations along the sequence dimension — unlike SP which only shards non-TP layers. CP uses Ring Attention: each GPU holds a chunk of tokens and computes attention in a ring, collecting K/V from other ranks. This is essential for training with context lengths of 32K, 128K, or longer.

SP vs. CP — Key Differences

Sequence Parallelism (SP):

- Shards activations of non-TP layers (LayerNorm, Dropout) along seq
- Requires TP to be enabled (replaces AllReduce with RS+AG)
- Does NOT shard attention KV cache

Context Parallelism (CP):

- Shards ALL layer activations (including attention) along seq
- Independent of TP (can be used with TP=1)
- Attention: Ring Attention — each rank holds seq/CP tokens
- AllGather for K/V across CP ring; local Q attends progressively

SP + CP: maximum activation memory savings for long-context training

```

# Context Parallelism for long-sequence training
python pretrain_gpt.py \
  --context-parallel-size 8 \           # 8 GPUs share 128K sequence
  --seq-length 131072 \               # 128K context!
  --tensor-model-parallel-size 4 \
  --sequence-parallel \               # combine SP + CP
  --pipeline-model-parallel-size 2 \
  # world_size = 4 × 8 × 2 × DP
  # Each CP GPU processes seq/8 = 16K tokens

# Dynamic CP (Megatron Core v0.12+) — adaptive sizing per batch
--dynamic-context-parallel-size      # up to 1.48x speedup for variable lengths

# TransformerConfig
config = TransformerConfig(
    context_parallel_size=8,
    sequence_parallel=True,
    # CP uses Ring Attention internally via core_attention implementation
)

```

3.5 Expert Parallelism (EP) for MoE

Expert Parallelism distributes MoE expert networks across GPUs. Each GPU owns E/EP experts. Token routing uses AllToAll dispatch: tokens travel to the GPU that owns their assigned expert, are processed, then travel back. EP is orthogonal to TP/PP/DP and only applies to expert layers.

```
# Expert Parallelism configuration
python pretrain_gpt.py \
  --num-experts 64 \           # 64 total experts
  --expert-model-parallel-size 8 \ # EP=8: each GPU holds 8 experts
  --moe-router-topk 2 \       # each token routes to 2 experts
  --moe-aux-loss-coeff 0.01 \  # load balancing loss weight
  --moe-z-loss-coeff 0.001 \   # router stability loss
  --moe-token-dispatcher-type alltoall \ # AllToAll for token routing
  --expert-tensor-parallel-size 2 \ # ETP: each expert split across 2 GPUs

# Shared experts (DeepSeek-V3 style): always-active + sparse-routed
--num-shared-moe-experts 2 \      # 2 always-active experts
--moe-shared-expert-intermediate-size 4096

# Full 5D parallelism with MoE:
# world_size = TP × PP × CP × EP × DP
# (EP replaces part of DP: each EP group processes different expert shards)
```

Router Strategy	Description
Top-K (default)	Each token activates K highest-probability experts. Simple, effective. K=2 typical.
Token Droplless	All tokens processed. Overloaded experts spill to buffer or skip. No info loss. Recommended.
Token Dropping	Excess tokens dropped when expert exceeds capacity. Capacity factor controls overflow.
Shared Experts	Some experts always active (like dense FFN). Others sparse. DeepSeek-V3 approach.
Expert Choice	Experts select their top-K tokens (reversed). Perfect load balance, non-uniform token assignment.

3.6 Distributed Data Parallel + Distributed Optimizer

Megatron's Distributed Optimizer (DistOpt) implements ZeRO-1: optimizer states (fp32 master weights, Adam m and v vectors) are sharded across DP ranks. Each rank only stores its own shard of optimizer states. Parameters and gradients are still replicated for the forward/backward pass.

DistOpt Memory Savings

Standard Adam mixed-precision per parameter:

2 bytes (fp16 param) + 4 (fp32 master) + 4 (m) + 4 (v) = 14 bytes

With DistOpt (DP=8):

2 bytes (param, replicated) + (4+4+4)/8 bytes (sharded opt) = 3.5 bytes

Savings: 14 → 3.5 bytes = 4x reduction in optimizer memory

For 7B model, DP=8: saves ~42 GB of optimizer state memory per GPU

```
# Enable Distributed Optimizer + communication overlaps
python pretrain_gpt.py \
  --use-distributed-optimizer \      # ZeRO-1 optimizer sharding
  --overlap-grad-reduce \           # overlap DP grad AllReduce w/ backward
```

```
--overlap-param-gather \           # overlap param AllGather w/ forward
--optimizer adam \
--adam-beta1 0.9 --adam-beta2 0.95 \
--adam-eps 1e-8 \
--weight-decay 0.1 \

# Optimizer CPU offload (saves GPU memory at cost of CPU<->GPU transfer)
--optimizer-cpu-offload
```

4. Architecture & Building Blocks (Megatron Core)

4.1 TransformerConfig — Central Configuration

TransformerConfig is the single configuration object that controls every aspect of a Megatron Core model. It is passed to all model constructors and training utilities. Centralising config avoids scattered argument parsing and enables clean composition.

```
from megatron.core.transformer.transformer_config import TransformerConfig
import torch

config = TransformerConfig(
    # — Architecture —
    num_layers=32,
    hidden_size=4096,
    num_attention_heads=32,
    num_query_groups=8,          # GQA: 8 KV heads, 32 Q heads
    ffn_hidden_size=16384,      # 4× hidden typical for dense
    max_position_embeddings=8192,
    seq_length=8192,

    # — Attention —
    attention_dropout=0.0,
    hidden_dropout=0.0,
    apply_rope_fusion=True,     # fused RoPE kernel via Apex
    rotary_base=500000,        # RoPE base (LLaMA3: 500000)
    q_lora_rank=None,           # MLA: query low-rank compression
    kv_lora_rank=None,          # MLA: key/value low-rank

    # — Normalization —
    normalization='RMSNorm',   # 'LayerNorm' | 'RMSNorm'
    norm_epsilon=1e-5,

    # — MLP / Activation —
    gated_linear_unit=True,     # SwiGLU (gate * silu(x))
    activation_func=torch.nn.functional.silu,

    # — Parallelism —
    tensor_model_parallel_size=8,
    pipeline_model_parallel_size=4,
    context_parallel_size=2,
    expert_model_parallel_size=8, # for MoE
    sequence_parallel=True,
    pipeline_dtype=torch.bfloat16,

    # — Precision —
    bf16=True,
    fp16=False,
    params_dtype=torch.bfloat16,
    fp8='hybrid',              # None | 'e4m3' | 'e5m2' | 'hybrid'
    fp8_amax_history_len=1024,
    fp8_amax_compute_algo='max',

    # — Memory Optimisations —
    recompute_granularity='selective', # 'full' | 'selective'
    recompute_method='uniform',
```

```

    use_flash_attn=True,
    async_tensor_model_parallel_allreduce=True,
    tp_comm_overlap=True,
)

```

4.2 ModuleSpec: Composable Layer System

Megatron Core's ModuleSpec system allows declaring transformer layer components declaratively. This separates architecture definition from backend choice (local PyTorch vs. Transformer Engine FP8). Swapping a spec from 'local' to 'transformer_engine' upgrades to FP8 without changing model logic.

```

from megatron.core.transformer.spec_utils import ModuleSpec, build_module
from megatron.core.transformer.transformer_layer import TransformerLayerSubmodules
from megatron.core.transformer.attention import SelfAttention, SelfAttentionSubmodules
from megatron.core.transformer.mlp import MLP, MLPSubmodules
from megatron.core.tensor_parallel.layers import ColumnParallelLinear, RowParallelLinear
from megatron.core.transformer.dot_product_attention import DotProductAttention

# — Local PyTorch spec —
local_spec = ModuleSpec(
    module=TransformerLayer,
    submodules=TransformerLayerSubmodules(
        self_attention=ModuleSpec(
            module=SelfAttention,
            submodules=SelfAttentionSubmodules(
                linear_qkv=ColumnParallelLinear,
                core_attention=DotProductAttention,
                linear_proj=RowParallelLinear,
            ),
        ),
        mlp=ModuleSpec(
            module=MLP,
            submodules=MLPSubmodules(
                linear_fc1=ColumnParallelLinear,
                linear_fc2=RowParallelLinear,
            ),
        ),
        pre_mlp_layernorm=RMSNorm,
        pre_self_attn_layernorm=RMSNorm,
    ),
)

# — Transformer Engine spec (FP8 on H100) —
# Swaps in TELinear, TEDotProductAttention, TENorm for all components
from megatron.core.models.gpt.gpt_layer_specs import (
    get_gpt_layer_local_spec,
    get_gpt_layer_with_transformer_engine_spec,
)
te_spec = get_gpt_layer_with_transformer_engine_spec()

# Build model with spec
from megatron.core.models.gpt.gpt_model import GPTModel
model = GPTModel(
    config=config,
    transformer_layer_spec=te_spec,  # swap local_spec / te_spec here
)

```

```

vocab_size=50257,
max_sequence_length=8192,
)

```

4.3 Supported Architectures

Architecture	Key Features & Notes
GPT (Decoder-only)	Causal attention mask, RoPE/ALiBi/no PE, GQA support, SwiGLU/GeGLU, RMSNorm. Default for LLMs.
BERT (Encoder-only)	Bidirectional attention, MLM+NSP. Embedding models, NLU classification tasks.
T5 (Encoder-Decoder)	Cross-attention between encoder/decoder. Relative position biases. Seq2Seq tasks.
Mamba / SSM Hybrid	State Space Model layers interleaved with attention. Linear attention alternative. Parallel scan.
MoE GPT	GPT backbone with MoE layers at configurable frequency. Top-K or shared expert routing.
Multi-Latent Attention (MLA)	DeepSeek-V3 style: Q,K,V low-rank compressed before projection. Reduces KV cache size.
Multi-Token Prediction (MTP)	Auxiliary heads predict multiple future tokens. Improves sample efficiency.
Multimodal (ViT + LLM)	Vision encoder (ViT) + language decoder. Image-text training via Megatron Energon.

5. Data Preparation Pipeline

5.1 Megatron's Indexed Dataset Format

Megatron uses a custom binary indexed dataset format for ultra-fast data loading. Raw text is tokenised once and stored in a memory-mapped `.bin` file; document offsets are stored in an `.idx` file. At training time, the DataLoader reads directly from the mmap file — no tokenisation overhead, $O(1)$ random access to any document.

5.2 Step 1 — Raw Data to Binary

```
# Step 1a: Download and clean data
# Common sources: The Pile, RedPajama, Dolma, C4, StarCoder, ROOTS
wget https://data.example.com/train.jsonl # each line: {"text": "..."}

# Step 1b: Tokenise and build binary index
python tools/preprocess_data.py \
  --input /data/raw/train.jsonl \
  --output-prefix /data/processed/my_dataset \
  --vocab-file /data/vocab/gpt2-vocab.json \
  --merge-file /data/vocab/gpt2-merges.txt \
  --tokenizer-type GPT2BPETokenizer \
  --dataset-impl mmap \
  --append-eod \
  --workers 32 \
  --log-interval 10000

# For LLaMA / Mistral / Qwen tokenizer (HuggingFace-compatible)
python tools/preprocess_data.py \
  --input /data/raw/train.jsonl \
  --output-prefix /data/processed/llama_dataset \
  --tokenizer-type HuggingFaceTokenizer \
  --tokenizer-model /models/llama3-tokenizer/ \
  --dataset-impl mmap \
  --append-eod

# Output files:
# /data/processed/my_dataset_text_document.bin (tokenised IDs, uint16/uint32)
# /data/processed/my_dataset_text_document.idx (doc start offsets + sizes)
```

5.3 Step 2 — Blended Multi-Source Dataset

```
# BlendedMegatronDatasetBuilder: mix sources with weights
from megatron.core.datasets.blended_megatron_dataset_builder import (
    BlendedMegatronDatasetBuilder
)
from megatron.core.datasets.gpt_dataset import GPTDataset, GPTDatasetConfig

config = GPTDatasetConfig(
    random_seed=42,
    sequence_length=4096,
    blend=[
        ('/data/web_text', 0.40), # 40% web text
```

```
        ('/data/code',          0.30), # 30% code
        ('/data/books',        0.20), # 20% books
        ('/data/wikipedia',    0.10), # 10% Wikipedia
    ],
    split='949,50,1',           # train/valid/test: 94.9/5.0/0.1%
    path_to_cache='/data/cache', # cache blending index for fast restarts
    reset_position_ids=False,
    reset_attention_mask=False,
    eod_mask_loss=True,         # mask loss on EOD tokens
)

train_ds, valid_ds, test_ds = BlendedMegatronDatasetBuilder(
    GPTDataset,
    sizes=[800_000_000, 50_000, 5_000], # num samples per split
    config=config,
).build()

# Equivalent via CLI
# --data-path 0.4 /data/web_text 0.3 /data/code 0.2 /data/books 0.1 /data/wikipedia
# --split 949,50,1
```

5.4 Batch Size Rampup

```
# Gradually increase global batch size from 16 to 2048 over 5M samples
python pretrain_gpt.py \
    --global-batch-size 2048 \
    --rampup-batch-size 16 16 5000000 \
    # start=16, increment=16, rampup_samples=5M
    # LR schedule counts in tokens; batch ramp is in samples
    # Useful for stable early training loss before scaling up batch
```

6. Complete Training Workflows

6.1 Pre-training 7B GPT on 8 GPUs (Single Node)

```
#!/bin/bash
# pretrain_gpt3_7b_8gpu.sh
# Model: 7B GPT | Hardware: 8x A100 80GB | Config: TP=4, PP=1, DP=2

CHECKPOINT_PATH=./checkpoints/gpt3_7b
VOCAB_FILE=./data/gpt2-vocab.json
MERGE_FILE=./data/gpt2-merges.txt
DATA_PATH='0.5 ./data/pile 0.5 ./data/code'

torchrun --standalone --nproc_per_node=8 pretrain_gpt.py \
# Architecture
--num-layers 32 --hidden-size 4096 --num-attention-heads 32 \
--seq-length 2048 --max-position-embeddings 2048 \
--ffn-hidden-size 16384 \
--swiglu \
--normalization RMSNorm \
--use-rotary-position-embeddings \

# Parallelism
--tensor-model-parallel-size 4 \
--pipeline-model-parallel-size 1 \
--sequence-parallel \

# Batch sizing
--micro-batch-size 2 \
--global-batch-size 1024 \

# Training
--train-iters 500000 \
--lr 1.5e-4 --lr-decay-style cosine --lr-warmup-iters 2000 \
--min-lr 1.5e-5 --lr-decay-iters 480000 \
--weight-decay 0.1 --clip-grad 1.0 \
--adam-betas 0.9 --adam-beta2 0.95 --adam-eps 1e-8 \

# Memory optimisations
--bf16 \
--use-distributed-optimizer \
--overlap-grad-reduce \
--overlap-param-gather \
--recompute-activations \
--use-flash-attn \

# Data
--vocab-file $VOCAB_FILE --merge-file $MERGE_FILE \
--data-path $DATA_PATH --split 949,50,1 \

# Checkpointing & logging
--save $CHECKPOINT_PATH --load $CHECKPOINT_PATH \
--save-interval 2000 --eval-interval 200 --eval-iters 100 \
--log-interval 100 --tensorboard-dir ./output/tb \
--log-throughput
```

6.2 Pre-training 175B GPT-3 on Multi-Node SLURM Cluster

```
#!/bin/bash
#SBATCH --job-name=gpt3_175b
#SBATCH --nodes=64          # 64 nodes × 8 GPUs = 512 GPUs total
#SBATCH --ntasks-per-node=1
#SBATCH --gres=gpu:8
#SBATCH --time=72:00:00

MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n1)
NODE_RANK=$SLURM_PROCID

# Config: TP=8 (intra-node NVLink), PP=16 (inter-node IB), DP=4
# world_size = 8 × 16 × 4 = 512 ✓
# Model replica: 8×16 = 128 GPUs
# Each PP stage: 96/16 = 6 layers on 8 TP GPUs

torchrun \
  --nproc_per_node=8 --nnodes=64 \
  --node_rank=$NODE_RANK \
  --master_addr=$MASTER_ADDR --master_port=29500 \
  pretrain_gpt.py \
  --num-layers 96 --hidden-size 12288 --num-attention-heads 96 \
  --seq-length 2048 --max-position-embeddings 2048 \

  --tensor-model-parallel-size 8 \
  --pipeline-model-parallel-size 16 \
  --num-layers-per-virtual-pipeline-stage 6 \ # VPP=6 for lower bubble
  --sequence-parallel \
  --tp-comm-overlap \

  --micro-batch-size 1 \
  --global-batch-size 1536 \
  --rampup-batch-size 16 16 5859375 \
  --train-samples 300000000000 \      # 300B tokens

  --bf16 \
  --use-distributed-optimizer \
  --overlap-grad-reduce --overlap-param-gather \
  --recompute-activations \
  --use-flash-attn \

  --lr 6e-5 --lr-decay-style cosine \
  --lr-warmup-samples 375000000 \
  --min-lr 6e-6 --clip-grad 1.0 \
  --weight-decay 0.1 \

  --vocab-file $VOCAB_FILE --merge-file $MERGE_FILE \
  --data-path $DATA_PATH \
  --save $CKPT --load $CKPT --save-interval 500
```

6.3 Megatron Core Python API Training Loop

```
import os, torch
import torch.distributed as dist
from megatron.core import parallel_state
```

```

from megatron.core.transformer.transformer_config import TransformerConfig
from megatron.core.models.gpt.gpt_model import GPTModel
from megatron.core.models.gpt.gpt_layer_specs import get_gpt_layer_local_spec
from megatron.core.pipeline_parallel.schedules import get_forward_backward_func
from megatron.core.distributed import DistributedDataParallelConfig
from megatron.core.distributed import DistributedDataParallel as MCoreDDP
from megatron.core.optimizer import get_megatron_optimizer, OptimizerConfig

# - 1. Init distributed -
dist.init_process_group(backend='nccl')
local_rank = int(os.environ.get('LOCAL_RANK', 0))
torch.cuda.set_device(local_rank)
parallel_state.initialize_model_parallel(
    tensor_model_parallel_size=4,
    pipeline_model_parallel_size=2,
    context_parallel_size=1,
)

# - 2. Build model -
config = TransformerConfig(
    num_layers=32, hidden_size=4096, num_attention_heads=32,
    tensor_model_parallel_size=4, pipeline_model_parallel_size=2,
    bf16=True, sequence_parallel=True,
)
model = GPTModel(config=config,
                  transformer_layer_spec=get_gpt_layer_local_spec(),
                  vocab_size=50257, max_sequence_length=2048)

# Wrap with DDP
ddp_config = DistributedDataParallelConfig(use_distributed_optimizer=True)
model = MCoreDDP(config=config, ddp_config=ddp_config, module=model)

# - 3. Optimizer -
opt_config = OptimizerConfig(
    optimizer='adam', lr=1e-4,
    adam_beta1=0.9, adam_beta2=0.95,
    weight_decay=0.1, bf16=True,
)
optimizer = get_megatron_optimizer(opt_config, [model])

# - 4. Forward step -
def forward_step(data_iter, model):
    batch = next(data_iter)
    tokens = batch['tokens'].cuda()
    labels = batch['labels'].cuda()
    attention_mask = batch['attention_mask'].cuda()
    position_ids = batch['position_ids'].cuda()
    loss_mask = batch['loss_mask'].cuda()
    out = model(tokens, position_ids, attention_mask, labels=labels)
    def loss_fn(x):
        loss = torch.sum(x.float().view(-1)*loss_mask.view(-1).float())/loss_mask.sum()
        return loss, {'lm loss': loss}
    return out, loss_fn

# - 5. Train loop -
fwd_bwd_fn = get_forward_backward_func()
for step in range(10000):
    model.zero_grad_buffer()

```

```
losses = fwd_bwd_fn(
    forward_step_func=forward_step,
    data_iterator=train_iter,
    model=model,
    num_microbatches=8,
    seq_length=2048,
    micro_batch_size=2,
    forward_only=False,
)
model.finish_grad_sync()
ok, grad_norm, _ = optimizer.step()
if step % 100 == 0 and dist.get_rank() == 0:
    print(f'Step {step}: loss={losses[0]["lm loss"].item():.4f}
gnorm={grad_norm:.3f}')
```

7. Mixed Precision & FP8 Training

7.1 Precision Formats

Format	Bits	Range	Usage in Megatron
fp32	32	Full	Master weights, optimizer states
bf16	16 (1+8+7)	Same as fp32	Default A100/H100. No loss scaling. Recommended.
fp16	16 (1+5+10)	±65504	Older GPUs. Needs GradScaler. Prone to NaN.
fp8-hybrid	8	Mixed	H100 via TE. e4m3 fwd + e5m2 bwd. Up to 2× vs bf16.
fp4 (experimental)	4	Very limited	Blackwell GPUs. Research quantisation.

7.2 FP8 Training (H100)

```
# FP8 training - requires H100 + Transformer Engine
python pretrain_gpt.py \
  --fp8-format hybrid \           # e4m3 fwd + e5m2 bwd
  --fp8-amax-history-len 1024 \   # rolling window for scaling factor
  --fp8-amax-compute-algo max \   # 'max' or 'most_recent'
  --fp8-margin 0 \
  --transformer-impl transformer_engine \ # CRITICAL for FP8
  --bf16 \                         # master weights still bf16
  --tensor-model-parallel-size 8 \
  --sequence-parallel

# TransformerConfig for FP8
config = TransformerConfig(
    fp8='hybrid',
    fp8_margin=0,
    fp8_interval=1,                # update scaling factors every iteration
    fp8_amax_history_len=1024,
    fp8_amax_compute_algo='max',
    fp8_wgrad=True,               # fp8 weight gradients
    bf16=True,
)
layer_spec = get_gpt_layer_with_transformer_engine_spec()
# TELinear/TEDotProductAttention/TENorm used instead of standard PyTorch layers
```

7.3 Activation Recomputation

Mode	Details
--recompute-activations (selective, default)	Recomputes only memory-intensive activations (QKV projections, softmax). ~5% FLOPs overhead, ~60% activation memory savings. RECOMMENDED.
--recompute-granularity full	Recomputes ALL activations of entire transformer layers. Maximum memory savings. For GPT-3: 70% activation memory reduction at 2.7% FLOPs cost.

Mode	Details
--recompute-method uniform	Spread recomputed layers uniformly across pipeline stages for balanced memory.
--recompute-method block	Exact N layers recomputed per stage via --recompute-num-layers N.
Fine-grained activation offload	CPU offload of specific activations. Pipeline-aware (Megatron Core v0.10+). Further reduces GPU peak memory.

8. Mixture of Experts (MoE) Training

8.1 MoE Configuration

```
# Full MoE config in TransformerConfig
moe_config = TransformerConfig(
    num_layers=32, hidden_size=4096, num_attention_heads=32,

    # MoE architecture
    num_moe_experts=64,                # total expert count
    moe_router_topk=2,                 # top-2 routing per token
    moe_ffn_hidden_size=2048,          # per-expert hidden size
    num_shared_moe_experts=2,          # always-active shared experts
    moe_shared_expert_intermediate_size=4096,

    # Load balancing
    moe_router_load_balancing_type='aux_loss',
    moe_aux_loss_coeff=0.01,
    moe_z_loss_coeff=0.001,
    moe_input_jitter_eps=0.01,

    # Expert frequency
    moe_layer_freq=1,                  # every layer is MoE

    # Token dispatch
    moe_token_dispatcher_type='alltoall',
    moe_expert_capacity_factor=None,    # None = token-droptless

    # Parallelism
    expert_model_parallel_size=8,
    expert_tensor_parallel_size=2,
    tensor_model_parallel_size=4,
    pipeline_model_parallel_size=4,
)
```

8.2 MoE Training Launch Command

```
# 30B MoE model (4B active params) - 128 GPUs: TP=4, PP=4, EP=8
torchrun --nproc_per_node=8 --nnodes=16 pretrain_gpt.py \
    --num-layers 28 --hidden-size 4096 --num-attention-heads 32 \
    --seq-length 4096 \

# Parallelism
--tensor-model-parallel-size 4 \
--pipeline-model-parallel-size 4 \
--expert-model-parallel-size 8 \
--sequence-parallel \
--tp-comm-overlap \

# MoE
--num-experts 64 \
--moe-router-topk 2 \
--moe-aux-loss-coeff 0.01 \
--moe-z-loss-coeff 0.001 \
```

```
--moe-token-dispatcher-type alltoall \  
  
# Training  
--bf16 \  
--use-distributed-optimizer \  
--overlap-grad-reduce \  
--recompute-activations \  
--micro-batch-size 1 \  
--global-batch-size 2048 \  
--train-samples 1000000000000 \ # 1T tokens  
  
--save $CKPT --load $CKPT --save-interval 500
```

9. Distributed Checkpointing

9.1 Checkpoint Formats

Format	When to Use
Native Megatron (.pt shards)	Default. Each TP/PP rank saves own shard. Requires same topology to load.
torch_dist format	PyTorch distributed checkpoint. Topology-independent. Recommended for production.
HuggingFace safetensors	Convert via Megatron Bridge for serving/inference. Single-GPU loadable.
TRT-LLM weight export	For TensorRT-LLM inference. Maximum deployment throughput.

9.2 Saving & Loading

```
# CLI: save every 500 iters, keep 3 most recent
python pretrain_gpt.py \
  --save /ckpt/gpt3_7b --load /ckpt/gpt3_7b \
  --save-interval 500 \
  --keep-last-n 3

# Python API: distributed checkpoint (topology-independent)
import megatron.core.dist_checkpointing as dist_ckpt

# Save
state = {
    'model': model.sharded_state_dict(),
    'optimizer': optimizer.sharded_state_dict(model.sharded_state_dict()),
    'rng_state': torch.get_rng_state(),
}
dist_ckpt.save(state, checkpoint_dir='/ckpt/step_50000')

# Load (even on different TP/PP topology!)
dist_ckpt.load(state, checkpoint_dir='/ckpt/step_50000')
model.load_state_dict(state['model'])
optimizer.load_state_dict(state['optimizer'])
```

9.3 Checkpoint Conversion

```
# Megatron -> HuggingFace (for serving/evaluation)
python tools/convert_checkpoint/megatron_to_transformers.py \
  --load-path /ckpt/gpt3_7b_tp4pp2 \
  --save-path /hf_ckpt/gpt3_7b \
  --target-tp 1 --target-pp 1

# HuggingFace -> Megatron (for continued large-scale training)
python tools/convert_checkpoint/transformers_to_megatron.py \
  --model-name meta-llama/Llama-3-8B \
  --save-path /ckpt/llama3_8b_megatron \
  --target-tp 4 --target-pp 2
```

```
# Reshape checkpoint topology
python tools/convert_checkpoint/megatron_to_megatron.py \
  --input-dir /ckpt/tp8_pp4 \
  --output-dir /ckpt/tp4_pp2 \
  --target-tensor-model-parallel-size 4 \
  --target-pipeline-model-parallel-size 2
```

10. Fine-tuning

10.1 Supervised Fine-tuning (SFT)

```
# SFT from pre-trained checkpoint
python pretrain_gpt.py \
  --finetune \                               # CRITICAL: resets iter count, reinits optimizer
  --load /ckpt/pretrained_7b \
  --save /ckpt/finetuned_7b \

# Same architecture as pre-trained
--num-layers 32 --hidden-size 4096 --num-attention-heads 32 \
--seq-length 4096 \

# Fine-tuning data (instruction pairs, JSONL format)
--data-path /data/alpaca_instruct \

# Lower LR, fewer iterations, no weight decay
--lr 1e-5 --train-iters 5000 --micro-batch-size 4 \
--global-batch-size 256 --weight-decay 0.0 \
--lr-warmup-iters 100 --lr-decay-style cosine \

--tensor-model-parallel-size 4 --bf16 --use-distributed-optimizer

# IMPORTANT: If fine-tuning is interrupted, REMOVE --finetune to resume
# With --finetune: always restarts from epoch 0
# Without --finetune: resumes from latest checkpoint (iter N)
```

10.2 Task Evaluation

```
# Zero-shot / few-shot evaluation on standard benchmarks
python tasks/run_text_generation_tasks.py \
  --task LambadaOpenAI \
  --load /ckpt/gpt3_7b \
  --temperature 0.0 --top_p 0.0 \
  --num-fewshot 0

# Supported tasks: LambadaOpenAI, RACE, MNLI, WinoGrande, BoolQ, ANLI

# Generate text (inference mode)
python tools/generate_samples_gpt.py \
  --load /ckpt/gpt3_7b \
  --num-samples 100 \
  --temperature 0.9 --top_p 0.95 \
  --out-seq-length 200
```

11. Industry Use Cases — How Big Companies Use Megatron-LM

11.1 MT-NLG 530B: Microsoft + NVIDIA (2021)

Configuration

Model: 530B parameters, 105 transformer layers, 20480 hidden size

Hardware: 560 DGX A100 servers = 4,480 A100 80GB GPUs

HDR InfiniBand full fat-tree (200 Gb/s per link)

NVIDIA Selene supercomputer + Microsoft Azure NDv4

Parallelism Strategy:

Tensor Parallelism (TP) = 8 — intra-node via NVLink

Pipeline Parallelism (PP) = 35 — inter-node via InfiniBand (DeepSpeed PP)

Data Parallelism (DP) — ZeRO-1 via DeepSpeed

Each model replica = $8 \times 35 = 280$ GPUs

Key insight: Megatron-LM TP within node (NVLink); DeepSpeed PP across nodes (IB)

Throughput: 126 TeraFLOPs/GPU sustained (A100 peak: 312 TF BF16)

Training: BF16 mixed precision + DeepSpeed ZeRO memory optimisation

11.2 BLOOM 176B: BigScience / HuggingFace (2022)

Configuration

Model: 176B parameters, 70 layers, 14336 hidden, 112 attention heads

Hardware: Jean Zay supercomputer (IDRIS/GENCI), 384 A100 80GB GPUs

Framework: Megatron-DeepSpeed fork (BigScience fork of NVIDIA repo)

From Megatron-LM: Tensor Parallelism (TP=4), BF16 activations, SP

From DeepSpeed: Pipeline Parallelism (PP=12), BF16 optimizer, ZeRO-1

Data: ROOTS corpus — 341B tokens, 59 natural languages

Training: 3.5 months, 1M+ GPU-hours

Peak: ~150 TFLOPs/GPU, ~43% MFU

Notable: First openly documented large-scale multilingual training run.

SP reduced activation memory enabling batch_size=1 per TP replica.

Opened all training logs, hyperparameters, and intermediate checkpoints publicly.

11.3 Typical Enterprise Pipeline (NeMo + Megatron Core)

```
# _____
# END-TO-END ENTERPRISE LLM WORKFLOW
# _____

# PHASE 1: DATA ENGINEERING
```

```
# -----
# 1. Collect: web crawl (CC), code (GitHub), books, papers
# 2. Deduplicate: MinHash LSH, exact URL dedup
# 3. Quality filter: perplexity threshold, heuristics (word ratio, etc)
# 4. Domain blend: define mix weights (40% web, 30% code, 20% books, 10% wiki)
# 5. Tokenise: tools/preprocess_data.py -> .bin/.idx
# 6. Validate: check token counts, EOS stats, vocabulary coverage

# PHASE 2: PRE-TRAINING
# -----
# 1. Config: TransformerConfig (TP/PP/CP/EP + precision)
# 2. Hardware: DGX cluster or cloud (Azure NDv4, AWS p4d, GCP a3)
# 3. Launch: SLURM srun torchrun OR NeMo launcher OR k8s job
# 4. Monitor: TensorBoard loss curves, grad norms, MFU, throughput
# 5. Checkpoint: every N iterations to distributed checkpoint store
# 6. Fault recovery: automatic restart from last checkpoint on node failure
# 7. Evaluation: periodic perplexity on valid set + few-shot benchmarks

# PHASE 3: POST-TRAINING / ALIGNMENT
# -----
# 1. Convert checkpoint: Megatron -> HuggingFace via Megatron Bridge
# 2. SFT: --finetune on instruction pairs (Alpaca, ShareGPT, custom)
# 3. Reward model: train on preference data pairs
# 4. RLHF/PPO: actor + ref + reward + critic training
# 5. DPO/GRPO: optional preference optimisation without RL

# PHASE 4: INFERENCE & DEPLOYMENT
# -----
# 1. Quantise: FP8 (H100) or INT8/INT4 via NVIDIA ModelOpt
# 2. TRT-LLM export: megatron.core.export.TRTLLMHelper
# 3. Triton server: serve via NVIDIA Triton Inference Server
# 4. NIM microservices: packaged, containerised inference API
# 5. Scale: horizontal with K8s + load balancer
```

11.4 Models Built on Megatron-LM

Model / Org	Scale	Key Megatron Features
Megatron 8.3B (NVIDIA, 2019)	8.3B, 512 V100	First TP: column/row parallel, 76% scaling efficiency vs 1 GPU
MT-NLG 530B (Microsoft+NVIDIA)	530B, 4480 A100	TP=8 (Megatron) + PP=35+ZeRO (DeepSpeed), 280 GPUs/replica
BLOOM 176B (BigScience)	176B, 384 A100	TP=4+PP=12+ZeRO-1, open training, Megatron-DeepSpeed fork
Megatron-LM 1T (NVIDIA, 2021)	1T, 3072 A100	3D parallel at scale: 502 PetaFLOPs/s, VPP introduced
Nemotron-H 56B (NVIDIA, 2025)	56B hybrid SSM	Mamba+attn hybrid, Megatron Core v0.12, full 3D parallel
DeepSeek-V3 (community)	671B MoE	EP+TP+PP, DeepEP, shared experts, MTP, FP8
Mixtral 8x7B / 8x22B	~46B active	EP=8, top-2 routing, token-droplless MoE

Model / Org	Scale	Key Megatron Features
Falcon-H1 (TII)	Custom hybrid	BitNet ternary weights, Megatron Bridge integration

12. Performance Optimisations & Communication Overlap

12.1 Communication Overlap Flags

Flag	What It Does
--overlap-grad-reduce	Overlaps DP gradient AllReduce with backward pass computation. Hides nearly all DP comm latency.
--overlap-param-gather	Overlaps parameter AllGather (DistOpt) with forward pass. Pre-fetches params while computing.
--tp-comm-overlap	Overlaps TP ReduceScatter/AllGather with next GEMM. Up to 30% TP comm hidden. Requires SP.
--async-tensor-model-parallel-allreduce	Async TP AllReduce overlapping with GeLU in col->row transition.
PP default double buffer	Pipeline activation tensors double-buffered. P2P comm overlaps with stage computation by default.

12.2 Fused Kernels

Kernel	Benefit
Fused LayerNorm (Apex)	Single CUDA kernel for forward+backward. Avoids extra memory reads/writes.
Fused Softmax	Combined causal mask + softmax in one kernel. Reduces memory traffic.
Fused Bias + GeLU / SwiGLU	Bias addition and activation fused. Avoids intermediate tensor allocation.
Flash Attention 2/3	O(N) memory attention via tiling. Critical for long sequences. Avoids materialising N×N attn matrix.
Fused RoPE	Rotary position embedding in single CUDA kernel (--apply-rope-fusion).
Fused Cross-Entropy	Loss computation + log-softmax fused. Avoids large [vocab_size] intermediate tensor.
Grouped GEMM (MoE)	TEGroupedLinear: all expert GEMMs in one batched kernel. Critical for MoE throughput.

12.3 Profiling & Key Metrics

```
# Built-in throughput logging
python pretrain_gpt.py \
  --log-throughput \
  --log-params-norm \
  --log-timers-to-tensorboard \
  --tensorboard-dir output/tensorboard

# Target metrics:
# MFU (Model FLOP Utilization): target > 40% on H100
```

```
# MFU = actual_throughput / theoretical_peak_flops
# TFLOP/s per GPU: Megatron achieves ~280-300 TF/s on A100, ~400 TF/s on H100
# Comm/compute ratio: < 20% for healthy training
# Expert utilisation entropy (MoE): high entropy = good load balance

# NVIDIA Nsight Systems profiling
nsys profile -w true -t cuda,nvtx \
  --capture-range=cudaProfilerApi \
  torchrun ... pretrain_gpt.py \
  --profile --profile-step-start 10 --profile-step-end 20 --profile-ranks 0

# Expected profiling insights:
# - NCCL AllReduce: should be overlapped (not blocking)
# - GEMM utilization: > 90% of compute budget
# - Data loading: < 5% of wall time
```

13. Quick Reference Cheat Sheet

13.1 Key CLI Arguments

Argument	Type	Description
--tensor-model-parallel-size N	int	TP degree. Intra-layer weight splitting.
--pipeline-model-parallel-size N	int	PP degree. Layer sequential distribution.
--context-parallel-size N	int	CP degree. Sequence Ring Attention.
--expert-model-parallel-size N	int	EP degree. Expert distribution.
--num-layers N	int	Total layers. Evenly split across PP stages.
--hidden-size H	int	Hidden dim. Divisible by TP and num-heads.
--num-attention-heads H	int	Total heads. Divisible by TP.
--num-query-groups G	int	KV heads for GQA. Divides num-heads.
--seq-length S	int	Training sequence length.
--micro-batch-size M	int	Per-GPU per-microbatch batch size.
--global-batch-size G	int	Total batch = $M \times DP \times \text{num_microbatches}$.
--bf16 / --fp16	flag	Mixed precision format.
--fp8-format hybrid	str	FP8 on H100. 'e4m3'/'e5m2'/'hybrid'.
--recompute-activations	flag	Selective activation recomputation.
--use-distributed-optimizer	flag	ZeRO-1 optimizer state sharding.
--sequence-parallel	flag	SP alongside TP.
--tp-comm-overlap	flag	Overlap TP comm with compute.
--overlap-grad-reduce	flag	Overlap DP grad AllReduce with backward.
--finetune	flag	Fine-tuning mode: reset iter, reinit opt.
--transformer-impl	str	'local' or 'transformer_engine' (FP8).

13.2 Parallelism Selection Guide

Rules of Thumb

1. TP first, fill NVLink node: TP=8 on 8-GPU nodes (NVLink). TP>8 crosses node = bad.
2. PP for depth, across nodes: PP distributes layers over nodes (InfiniBand acceptable).
3. CP for long context: Enable CP when seq_length > 8K and KV cache is bottleneck.
4. EP for MoE: EP = num_experts/local_experts (integer).
5. DP fills remaining GPUs: DP = world_size / (TP × PP × CP).
6. num_microbatches >> PP: Keep m ≥ 4×PP to reduce bubble to < 20%.

Example 70B on 64 GPUs (8 nodes × 8 GPUs):

TP=8, PP=4, DP=2 => 8×4×2=64 ✓

Each PP stage: 80 layers / 4 = 20 layers on 8 TP GPUs = 2.5B params/GPU

Example 405B on 256 GPUs (32 nodes × 8 GPUs):

TP=8, PP=8, DP=4 => 8×8×4=256 ✓

Each PP stage: 126 layers / 8 = ~16 layers on 8 TP GPUs = 6.3B params/GPU

13.3 Key Module APIs

Module	Key Classes / Functions
megatron.core.parallel_state	initialize_model_parallel(), get_tensor/pipeline/data_model_parallel_group()
megatron.core.transformer	TransformerConfig, TransformerLayer, TransformerBlock, MoELayer
megatron.core.models.gpt	GPTModel, get_gpt_layer_local_spec(), get_gpt_layer_with_transformer_engine_spec()
megatron.core.tensor_parallel	ColumnParallelLinear, RowParallelLinear, VocabParallelEmbedding
megatron.core.pipeline_parallel	get_forward_backward_func(), P2PCommunicator, forward_backward_pipelining_with_interleaving
megatron.core.distributed	DistributedDataParallel (MCore), DistributedDataParallelConfig
megatron.core.optimizer	get_megatron_optimizer(), DistributedOptimizer, OptimizerConfig
megatron.core.datasets	GPTDataset, BlendedMegatronDatasetBuilder, MockGPTDataset
megatron.core.dist_checkpointing	save(), load(), ShardedStateDictProtocol
megatron.core.inference	DynamicInferenceEngine, StaticInferenceEngine
megatron.core.extensions	TELinear, TEDotProductAttention, TENorm (FP8 layers)
megatron.core.fusions	fused_bias_gelu(), FusedLayerNorm, FusedRMSNorm
megatron.core.export	TRTLLMHelper, ExportConfig (TRT-LLM export)

13.4 Resources & Further Reading

- **GitHub:** github.com/NVIDIA/Megatron-LM — source, scripts, examples, MoE roadmap
- **Megatron Core Docs:** docs.nvidia.com/megatron-core — API reference, quickstart, parallelism guide
- **Paper: TP:** arXiv:1909.08053 — Megatron-LM: Training Multi-Billion Parameter LMs
- **Paper: PP+3D:** arXiv:2104.04473 — Efficient Large-Scale LM Training (Interleaved 1F1B)

- **Paper: SP:** [arXiv:2205.05198](https://arxiv.org/abs/2205.05198) — Reducing Activation Recomputation in Large Transformers
- **BLOOM Training Blog:** huggingface.co/blog/bloom-megatron-deepspeed — 176B step-by-step
- **MT-NLG Blog:** microsoft.com/research — DeepSpeed + Megatron 530B model
- **Megatron Bridge:** docs.nvidia.com/nemo/megatron-bridge — HF<->Megatron conversion
- **NeMo Framework:** developer.nvidia.com/nemo — enterprise training on Megatron Core
- **DeepWiki:** deepwiki.com/NVIDIA/Megatron-LM — AI-generated codebase walkthrough